

# Industrial Quality Assistant

---

## *Phase 2 Tutorial Report v4.0 | Beginner rebuild guide*

Document type: Step by step tutorial with code snippets, commands, diagrams, checks and screenshots references.

Purpose: enable a beginner developer to rebuild the Phase 2 API, replay and AI security workstream without relying on ChatGPT.

Scope: IQA2\_NAT01 to IQA2\_NAT17, IQA2\_JAL02, IQA2\_JAL03, IQA2\_JAL04 and the IQA2\_LIV03 notebook update scope.

Status: Final Phase 2 tutorial rewrite after JAL02, JAL03 and JAL04 validation. Kubernetes remains a Phase 4 perspective, while Kong API Gateway is a Phase 3 item.

**Important constraint:** All body text, tables and code snippets use font size 10 pt or larger. The diagrams are Word native tables or large text flows so they remain readable and editable.

## Table of contents

1. How to use this tutorial
2. Reference architecture and repo map
3. Environment setup and baseline validation
4. NAT01 to NAT17 rebuild guide
5. JAL02 live server validation
6. JAL03 and JAL04 validation gates
7. Final notebook update checklist
8. Troubleshooting
9. Evidence register and command cheat sheet

## 1. How to use this tutorial

This is not only an audit summary. Each task is written as a rebuild recipe: why the task exists, what files to modify, what code shape to implement, how to test it and what evidence proves it is done.

| Beginner action   | File or command               | What to check                                        | Expected result              |
|-------------------|-------------------------------|------------------------------------------------------|------------------------------|
| Read the goal     | NAT or JAL section            | Understand what protection or feature is being added | You know why the task exists |
| Edit the files    | src/iqa, tests, docs          | Use the code snippets as implementation templates    | Files compile                |
| Run focused tests | python -m pytest ... -q       | Fix only the failing contract                        | Targeted tests pass          |
| Run baseline      | python -m pytest tests -q     | Check no regression                                  | Baseline passes              |
| Record evidence   | planning comment and log file | Keep proof for deliverables                          | Task can move to 100%        |

### Beginner workflow

| Plan | Code | Test | Validate server | Document | Close task |
|------|------|------|-----------------|----------|------------|
|------|------|------|-----------------|----------|------------|

**Rule:** A task is not closed when code is written. It is closed when the code, tests, server evidence and planning comment are aligned.

## 2. Reference architecture and repository map

The IQA repository is organized as a small MLOps platform. The GitHub repository contains folders for workflows, Airflow DAGs, configs, data manifests, deployment, docs, scripts, source code and tests. The README describes IQA as an MLOps MVP for visual quality control on Casting parts, with PostgreSQL as the metadata target and MinIO for ingested images and artifacts.

| Folder            | Purpose                                    | Used in Phase 2 for                                          |
|-------------------|--------------------------------------------|--------------------------------------------------------------|
| .github/workflows | CI definitions                             | Future Phase 3 CI/CD consolidation                           |
| airflow/dags      | Airflow orchestration                      | Replay and lifecycle DAGs                                    |
| configs           | Runtime scenarios and replay configs       | production_replay_natural                                    |
| data/metadata     | Small metadata manifests                   | replay plans, piece events, calibration and scenario records |
| deploy            | Docker Compose and service definitions     | server stack and PostgreSQL runtime                          |
| docs              | Architecture, ADRs and governance docs     | NAT17 documentation deliverable                              |
| scripts           | Batch entry points and operational helpers | replay, lifecycle, database init                             |
| src/iqa/api       | FastAPI app and schemas                    | prediction, feedback, reload, metrics, incidents             |
| src/iqa/metadata  | metadata persistence abstraction           | Memory and PostgreSQL repositories                           |
| tests             | pytest contracts                           | all NAT and JAL validations                                  |

### Phase 2 runtime flow

| Streamlit / replay | FastAPI governance | Inference service | PostgreSQL | Metrics / incidents |
|--------------------|--------------------|-------------------|------------|---------------------|
|--------------------|--------------------|-------------------|------------|---------------------|

|  |                 |  |                 |  |
|--|-----------------|--|-----------------|--|
|  | <b>boundary</b> |  | <b>metadata</b> |  |
|--|-----------------|--|-----------------|--|

Correct governance rule: the inference service returns model output to FastAPI. FastAPI remains responsible for validation, incidents, metrics and metadata writes. This avoids bypassing API contracts.

## 3. Environment setup and baseline validation

### 3.1 Local development setup on CubeAI

```
cd /mnt/nvme/mlops/iqa-mlops
python --version
uv sync --extra cpu
uv pip install "psycpg[binary]>=3.2"
python - <<'PY'
import psycpg
print("PSYCOPG_OK", psycpg.__version__)
PY
```

| Beginner action  | File or command           | What to check                       | Expected result                              |
|------------------|---------------------------|-------------------------------------|----------------------------------------------|
| Check branch     | git status -sb            | You are on the expected work branch | No unrelated modified files                  |
| Synchronize main | git fetch origin --prune  | origin/main is reachable            | Fetch succeeds                               |
| Run full tests   | python -m pytest tests -q | No failing contract                 | All tests pass or live DB tests skip locally |

### 3.2 Server validation setup

When runtime state matters, validate from a detached worktree on the shared IQA server. This avoids modifying the live checkout.

```
REPO="/opt/iqa/iqa-mlops"
WT="/tmp/iqa_phase2_validation_$(date +%Y%m%d_%H%M%S)"
cd "$REPO"
git fetch origin --prune
git worktree add --detach "$WT" origin/main
cd "$WT"
PYTHONPATH="$WT/src" "$REPO/.venv/bin/python" -m pytest tests/api tests/security
tests/contracts -q
```

**Expected evidence:** For the final JAL02 live PostgreSQL validation, the proof was POSTGRES\_CONNECTION\_OK, schema initialized and 9 passed in 6.14 seconds.

## 4. NAT01 to NAT17 rebuild guide

Each NAT below is written so a beginner can rebuild the feature from a clean branch. The code blocks are implementation templates aligned with the final repository design. When the final repo already contains more complete code, keep the repo implementation and use this tutorial to understand how to rebuild it.

**How to read the code blocks:** The snippets are not only commands. They show the Python classes, Pydantic models, FastAPI guards, repository methods and pytest patterns that a beginner needs in order to recreate the work. After copying a snippet, adapt imports and names to the current repo layout, then run the focused test.

### IQA2\_NAT01 - Metadata repository foundation

| Beginner action  | File or command                                                          | What to check                                     | Expected result       |
|------------------|--------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Create a persistence boundary before choosing PostgreSQL.                | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/metadata/repository.py                                           | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/contracts/test_metadata_repository_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT01 comment                                                       | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```

from copy import deepcopy
from typing import Any, Protocol

class MetadataRepository(Protocol):
    def save_prediction(self, prediction_id: str, record: dict[str, Any]) -> None: ...
    def get_prediction(self, prediction_id: str) -> dict[str, Any] | None: ...
    def list_predictions(self) -> list[dict[str, Any]]: ...
    def save_feedback(self, prediction_id: str, record: dict[str, Any]) -> None: ...
    def get_feedback(self, prediction_id: str) -> dict[str, Any] | None: ...

class MemoryMetadataRepository:
    def __init__(self) -> None:
        self._predictions: dict[str, dict[str, Any]] = {}
        self._feedbacks: dict[str, dict[str, Any]] = {}

    def save_prediction(self, prediction_id: str, record: dict[str, Any]) -> None:
        self._predictions[prediction_id] = deepcopy(record)

    def get_prediction(self, prediction_id: str) -> dict[str, Any] | None:
        record = self._predictions.get(prediction_id)
        return deepcopy(record) if record is not None else None
  
```

**Validation command**

```
python -m pytest tests/contracts/test_metadata_repository_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT02 - Prediction traceability contract

| Beginner action  | File or command                                                     | What to check                                     | Expected result       |
|------------------|---------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Make every prediction auditable from image to model result.         | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/schemas.py                                              | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_predict_traceability_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT02 comment                                                  | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
from pydantic import BaseModel, Field

class PredictRequest(BaseModel):
    image_uri: str
    scenario_id: str = Field(min_length=1)
    piece_event_id: str | None = None
    source_event_id: str | None = None
    sha256: str | None = None
    lot_id: str | None = None
    source_class: str | None = None
    dataset_version: str | None = None
    replay_id: str | None = None
    validation_id: str | None = None

class PredictResponse(BaseModel):
    prediction_id: str
    scenario_id: str
    decision: str
    model_version: str | None = None
    roi_model_version: str | None = None
    feedback_closed: bool = False
```

### Validation command

```
python -m pytest tests/api/test_predict_traceability_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.



## IQA2\_NAT03 - Scenario scoped API

| Beginner action  | File or command                                            | What to check                                     | Expected result       |
|------------------|------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Prevent model, replay and drift scenario collisions.       | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py and schemas.py                         | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_scenario_id_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT03 comment                                         | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
def registered_model_name_for_scenario(scenario_id: str) -> str:
    if not scenario_id:
        raise ValueError("scenario_id is required")
    return f"feature_ae__{scenario_id}"

@app.post("/predict")
def predict(request: PredictRequest) -> PredictResponse:
    model_name = registered_model_name_for_scenario(request.scenario_id)
    # use model_name when querying MLflow or building response metadata
    ...
```

### Validation command

```
python -m pytest tests/api/test_scenario_id_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT04 - Feedback contract foundation

| Beginner action  | File or command                                                            | What to check                                     | Expected result       |
|------------------|----------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Create an explicit feedback request instead of free form comments.         | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/schemas.py                                                     | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_feedback_poisoning_controls_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT04 comment                                                         | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
class FeedbackRequest(BaseModel):
    prediction_id: str
    piece_event_id: str | None = None
    scenario_id: str
    feedback_source: str = "oracle_gt"
    status: str
    comment: str | None = None
    eligible_for_train: bool | None = None
```

### Validation command

```
python -m pytest tests/api/test_feedback_poisoning_controls_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT05 - human\_sophie display only feedback

| Beginner action  | File or command                                                               | What to check                                     | Expected result       |
|------------------|-------------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Allow Sophie review without turning it into training data.                    | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                                           | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_human_sophie_oracle_divergence_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT05 comment                                                            | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
if feedback.feedback_source == "human_sophie":
    display_record = feedback.model_dump()
    display_record["eligible_for_train"] = False
    display_record["display_only"] = True
    METADATA_REPOSITORY.save_display_feedback(prediction_id, display_record)
    return {"status": "display_feedback_recorded", "eligible_for_train": False}
```

### Validation command

```
python -m pytest tests/api/test_human_sophie_oracle_divergence_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT06 - oracle\_gt sovereign feedback

| Beginner action  | File or command                                                                       | What to check                                     | Expected result       |
|------------------|---------------------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Only trusted oracle ground truth can close feedback and compute training eligibility. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                                                   | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_train_eligibility_blocks_contract.py -q               | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT06 comment                                                                    | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
if feedback.feedback_source == "oracle_gt":
    eligible = compute_train_eligibility(prediction, feedback)
    record = feedback.model_dump()
    record["eligible_for_train"] = eligible
    METADATA_REPOSITORY.save_feedback(prediction_id, record)
    METADATA_REPOSITORY.mark_feedback_closed(prediction_id, closed_at=utc_now())
    return {"status": "feedback_closed", "eligible_for_train": eligible}
```

### Validation command

```
python -m pytest tests/api/test_train_eligibility_blocks_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT07 - Train eligibility safeguards

| Beginner action  | File or command                                                         | What to check                                     | Expected result       |
|------------------|-------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Block unsafe samples from entering training.                            | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/feedback/oracle.py                                              | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_train_eligibility_blocks_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT07 comment                                                      | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
def compute_train_eligibility(prediction: dict, feedback: dict) -> bool:
    if feedback.get("status") != "conforme":
        return False
    if prediction.get("decision") == "false_negative":
        return False
    if prediction.get("roi_status") in {"roi_warning", "roi_fail"}:
        return False
    if prediction.get("dataset_role") == "validation_set_v001":
        return False
    return True
```

### Validation command

```
python -m pytest tests/api/test_train_eligibility_blocks_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT08 - Admin reload governance

| Beginner action  | File or command                                                 | What to check                                     | Expected result       |
|------------------|-----------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Protect model reload operations with token, scenario and audit. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                             | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_api_contract_hardening.py -q    | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT08 comment                                              | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
def require_admin_token(authorization: str | None) -> None:
    expected = os.getenv("IQA_ADMIN_TOKEN")
    if not expected or authorization != f"Bearer {expected}":
        raise HTTPException(status_code=401, detail={"code": "unauthorized"})

@app.post("/admin/reload-model")
def reload_model(request: ReloadRequest, authorization: str | None = Header(None)):
    require_admin_token(authorization)
    record = {"scenario_id": request.scenario_id, "stage": request.stage, "accepted": True}
    METADATA_REPOSITORY.save_admin_reload_event(record)
    return {"status": "reload_requested"}
```

### Validation command

```
python -m pytest tests/api/test_api_contract_hardening.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT09 - Audit trail chain

| Beginner action  | File or command                                                      | What to check                                     | Expected result       |
|------------------|----------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Link sha256, piece event, scenario, lot, model version and feedback. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                                  | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_audit_trail_contract.py -q           | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT09 comment                                                   | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
prediction_record = {
    "prediction_id": prediction_id,
    "sha256": request.sha256,
    "piece_event_id": request.piece_event_id,
    "scenario_id": request.scenario_id,
    "lot_id": request.lot_id,
    "model_version": model_version,
    "roi_model_version": roi_model_version,
    "feedback_closed": False,
}
METADATA_REPOSITORY.save_prediction(prediction_id, prediction_record)
```

### Validation command

```
python -m pytest tests/api/test_audit_trail_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT10 - Prediction and lot history APIs

| Beginner action  | File or command                                                        | What to check                                     | Expected result       |
|------------------|------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Expose prediction history and lot summaries for review and governance. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                                    | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_filtered_metrics_contract.py -q        | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT10 comment                                                     | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
@app.get("/predictions")
def list_predictions():
    return {"items": METADATA_REPOSITORY.list_predictions()}

@app.get("/lots/summary")
def lots_summary():
    rows = METADATA_REPOSITORY.list_predictions()
    # group by lot_id and scenario_id for review dashboards
    return {"items": build_lot_summary(rows)}
```

### Validation command

```
python -m pytest tests/api/test_filtered_metrics_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.



## IQA2\_NAT11 - Filtered metrics

| Beginner action  | File or command                                                 | What to check                                     | Expected result       |
|------------------|-----------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Make security and quality status visible to Prometheus.         | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                             | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_filtered_metrics_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT11 comment                                              | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
from prometheus_client import Counter

prediction_counter = Counter(
    "iqa_prediction_filtered_total",
    "Predictions by scenario, lot, class and model",
    ["scenario_id", "lot_id", "source_class", "model_version"],
)

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
```

### Validation command

```
python -m pytest tests/api/test_filtered_metrics_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT12 - Strict Phase 2 Pydantic schemas

| Beginner action  | File or command                                                        | What to check                                     | Expected result       |
|------------------|------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Normalize API inputs and outputs before security hardening.            | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/schemas.py                                                 | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_phase2_pydantic_schemas_contract.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT12 comment                                                     | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
class ApiError(BaseModel):
    code: str
    message: str
    details: dict[str, str] = {}

class IncidentRecord(BaseModel):
    incident_id: str
    incident_type: str
    severity: str
    scenario_id: str | None = None
    prediction_id: str | None = None
```

### Validation command

```
python -m pytest tests/api/test_phase2_pydantic_schemas_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT13 - Standardized API errors

| Beginner action  | File or command                                                             | What to check                                     | Expected result       |
|------------------|-----------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Return predictable errors for 400, 401, 403, 404, 409, 422 and 500 classes. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                                         | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_api_error_standardization_contract.py -q    | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT13 comment                                                          | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
def api_error(status_code: int, code: str, message: str, **details):
    raise HTTPException(
        status_code=status_code,
        detail={"code": code, "message": message, "details": details},
    )

if prediction is None:
    api_error(404, "prediction_not_found", "Prediction id was not found")
```

### Validation command

```
python -m pytest tests/api/test_api_error_standardization_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT14 - Incidents API

| Beginner action  | File or command                                                                           | What to check                                     | Expected result       |
|------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Create structured incidents for false negatives, ROI fail, conflicts and reload refusals. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | src/iqa/api/main.py                                                                       | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api/test_incidents_contract.py -q                                  | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT14 comment                                                                        | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
def create_incident(incident_type: str, severity: str, payload: dict) -> dict:
    incident = {
        "incident_id": f"inc_{uuid4().hex[:12]}",
        "incident_type": incident_type,
        "severity": severity,
        "payload": payload,
    }
    INCIDENT_STORE.append(incident)
    return incident

@app.get("/incidents")
def list_incidents():
    return {"items": INCIDENT_STORE}
```

### Validation command

```
python -m pytest tests/api/test_incidents_contract.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT15 - Contract tests reinforcement

| Beginner action  | File or command                                | What to check                                     | Expected result       |
|------------------|------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Lock the API behavior before final validation. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | tests/api and tests/contracts                  | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api tests/contracts -q  | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT15 comment                             | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
def test_unknown_prediction_returns_404(client):
    response = client.post("/feedback", json={"prediction_id": "missing", "scenario_id":
"production_replay_natural", "status": "conforme"})
    assert response.status_code == 404
    assert response.json()["detail"]["code"] == "prediction_not_found"
```

### Validation command

```
python -m pytest tests/api tests/contracts -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT16 - AI security governance tests

| Beginner action  | File or command                                                             | What to check                                     | Expected result       |
|------------------|-----------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Prove the threat model controls are executable tests.                       | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | tests/security/test_ai_security_governance_contracts.py                     | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/security/test_ai_security_governance_contracts.py -q | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT16 comment                                                          | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
def test_oracle_gt_is_sovereign_for_training_eligibility():
    human_feedback = {"feedback_source": "human_sophie", "status": "conforme"}
    assert human_feedback.get("eligible_for_train") is not True

    oracle_feedback = {"feedback_source": "oracle_gt", "status": "conforme"}
    assert oracle_feedback["feedback_source"] == "oracle_gt"
```

### Validation command

```
python -m pytest tests/security/test_ai_security_governance_contracts.py -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## IQA2\_NAT17 - Documentation and handover

| Beginner action  | File or command                                                          | What to check                                     | Expected result       |
|------------------|--------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
| Understand goal  | Document contracts, feedback rules, security governance and audit trail. | This task solves a concrete governance or API gap | Goal is clear         |
| Edit file(s)     | docs/*.md                                                                | Create or update the listed module(s)             | Code imports cleanly  |
| Add focused test | python -m pytest tests/api tests/security tests/contracts -q             | The contract describes expected behavior          | Focused test passes   |
| Update planning  | IQA2_NAT17 comment                                                       | Add proof and status                              | Task can move to 100% |

### Code block to implement or understand

```
docs/api_contracts.md
docs/feedback_rules.md
docs/ai_security_governance.md
docs/audit_trail.md
docs/index.md
```

### Validation command

```
python -m pytest tests/api tests/security tests/contracts -q
```

**Expected result:** The focused test passes. Then run the broader API and security baseline before closing the task.

## 5. JAL02 live server validation

JAL02 validates that production\_replay\_natural goes through the API, feedback, reload events and the PostgreSQL metadata store. This must be validated on the shared server, not only on CubeAI.

### JAL02 validated runtime flow

|                           |                  |                  |                     |                     |
|---------------------------|------------------|------------------|---------------------|---------------------|
| production_replay_natural | FastAPI /predict | Feedback /reload | PostgreSQL metadata | 9 live tests passed |
|---------------------------|------------------|------------------|---------------------|---------------------|

```
REPO="/opt/iqa/iqa-mlops"
WT="/tmp/iqa_jal02_live_postgres_$(date +%Y%m%d_%H%M%S)"
cd "$REPO"
git fetch origin --prune
git worktree add --detach "$WT" origin/main
cd "$WT"
set -a
. "$REPO/.env"
set +a
```

```
export IQA_METADATA_DB_URL="${IQA_METADATA_DB_URL}/@postgres:5432/@127.0.0.1:5432}"
PYTHONPATH="$WT/src" "$REPO/.venv/bin/python" scripts/init_metadata_db.py
PYTHONPATH="$WT/src" "$REPO/.venv/bin/python" -m pytest
tests/api/test_postgres_write_through_contract.py
tests/metadata/test_postgres_metadata_foundation.py -q -rs
```

**Expected result:** POSTGRES\_CONNECTION\_OK iqa\_metadata iqa, IQA metadata schema is initialized, and 9 passed.

## 6. JAL03 and JAL04 validation gates

JAL03 and JAL04 are validated Phase 2 gates. They should be described as completed in notebooks and livrables, not pending work.

| Gate  | Meaning                                                   | What to show                                           | Proof to keep                  |
|-------|-----------------------------------------------------------|--------------------------------------------------------|--------------------------------|
| JAL03 | Model lifecycle and promotion logic validated for Phase 2 | Feature AE candidate, MLflow registry, promotion rules | tests, docs, server status     |
| JAL04 | Platform delivery validation completed for Phase 2        | Docker stack, services, evidence, docs                 | planning, screenshots, reports |

## 7. Final notebook update checklist for IQA2\_LIV03

The remaining IQA2\_LIV03 work is the notebook refresh. A beginner should update notebooks so they tell the final truth of Phase 2.

- Replace every old JAL02 pending statement with JAL02 validated at 100% on server side PostgreSQL live.
- Add JAL03 and JAL04 as validated gates.
- Add Phase 2 end architecture and repo architecture.
- Add commands used for server validation.
- Add evidence cells with the exact output: POSTGRES\_CONNECTION\_OK, schema initialized and 9 passed.
- Move Kubernetes to Phase 4 unless the mentor explicitly asks for a Phase 3 demo.
- Describe Kong API Gateway as a Phase 3 task.

## 8. Troubleshooting

| Symptom                         | Cause                                                | Fix                                                                                                         |
|---------------------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| psycopg import fails            | Local .venv not synced after PostgreSQL merge        | uv pip install "psycopg[binary]>=3.2"                                                                       |
| IQA_METADATA_DB_URL is not set  | The .env file was not sourced                        | set -a; . .env; set +a                                                                                      |
| failed to resolve host postgres | Host name works inside Docker, not from server shell | Replace @postgres:5432 with @127.0.0.1:5432 for host tests                                                  |
| CSV files modified after tests  | Some tests regenerate Feature AE datasets            | git restore<br>data/model_datasets/feature_ae_good_v002.csv<br>data/model_datasets/feature_ae_good_v003.csv |



|                                       |                                                 |                                                  |
|---------------------------------------|-------------------------------------------------|--------------------------------------------------|
| PowerPoint or report images too small | Diagram was generated as bitmap with small text | Prefer Word native tables and 10 pt minimum font |
|---------------------------------------|-------------------------------------------------|--------------------------------------------------|

## 9. Evidence register and command cheat sheet

Use this page when preparing handover, report updates or defense evidence.

| Evidence                                   | Meaning                                    | Where to use it            |
|--------------------------------------------|--------------------------------------------|----------------------------|
| 464 passed, 11 skipped                     | Full CubeAI baseline after resync          | Phase 2 validation summary |
| POSTGRES_CONNECTION_OK<br>iqa_metadata iqa | Server PostgreSQL reachable                | JAL02 closure              |
| IQA metadata schema is initialized         | Metadata schema exists                     | JAL02 live evidence        |
| 9 passed in 6.14s                          | Live PostgreSQL contract tests passed      | JAL02 final proof          |
| JAL03 validated                            | Model lifecycle gate closed                | Notebooks and reports      |
| JAL04 validated                            | Platform delivery gate closed              | Notebooks and reports      |
| Kong API Gateway in Phase 3                | Gateway selected as industrialization task | Phase 3 planning           |
| Kubernetes in Phase 4                      | Avoid overengineering Phase 3              | Mentor discussion          |

### Command cheat sheet

```
# Local full baseline
python -m pytest tests -q

# API and security baseline
python -m pytest tests/api tests/security tests/contracts -q

# JAL02 live PostgreSQL tests
python -m pytest tests/api/test_postgres_write_through_contract.py
tests/metadata/test_postgres_metadata_foundation.py -q -rs

# Restore generated datasets after tests
git restore data/model_datasets/feature_ae_good_v002.csv
data/model_datasets/feature_ae_good_v003.csv

# Check branch state
git status -sb
git log --oneline -10 origin/main
```

## 10. What this tutorial enables

A beginner can now rebuild the Phase 2 workstream by following the task order, recreating the files, copying the code blocks, running the focused tests and comparing outputs to the expected evidence. The document does not hide implementation inside scripts only. Scripts are still used for runtime validation, but the tutorial also explains the code structure that those tests exercise.

**Final status:** NAT01 to NAT17, JAL02, JAL03 and JAL04 are covered. IQA2\_LIV03 remains a notebook update task, not an implementation gap.